

Assignment: Arithmetic Expression Tokenizer and Parser

CS2201: Formal Language And Automata Theory

IIT Patna

April 12, 2026

1 Objective

Implement a program using **LEX (Flex)** and **YACC (Bison)** that performs lexical analysis and syntax analysis on arithmetic expressions. The program must generate a token table for all identified lexemes, verify syntax, and evaluate the result of the expression.

2 Requirements

2.1 Lexical Analysis (LEX)

The LEX program must identify the following lexemes and print a **Token Table** (in the same order as input) before passing them to the parser:

- **NUM:** Integer or floating-point numbers.
- **OPERATORS:**
 - + → PLUS
 - - → MINUS
 - * → MULT
 - / → DIV
 - % → MOD
- **PARENTHESES:**
 - (→ LPAREN
 -) → RPAREN
- **WHITESPACE:** Must be ignored and **must not appear in the token table**.

2.2 Syntax Analysis & Evaluation (YACC)

The YACC program must enforce standard mathematical precedence and evaluate the expression:

- **Operators:** Support addition, subtraction, multiplication, division and modulo.
- **Precedence:**
 - Highest: $*$, $/$, $%$
 - Lowest: $+$, $-$
- **Grouping:** Correct handling of parentheses $()$.
- **Unary Minus:** Expressions like $-5 + 3$ must be supported.
- **Unary Operators:** Unary $+$ and $-$ must be supported. Multiple consecutive unary operators are allowed and must be evaluated right-to-left. For example: $--5 = 5$, $++-3 = 3$.
- **Modulo Constraint:** The $%$ operator is valid only when both operands are mathematically integers. An operand is considered an integer if its evaluated value has no fractional part (e.g., 5, 5.0, or $(10/2)$).

Examples:

- $5 \% 2 = 1$
- $5.0 \% 2 = 1$
- $(10/2) \% 3 = 2$
- $5.5 \% 2 \rightarrow \text{Invalid Expression}$
- $(10/3) \% 2 \rightarrow \text{Invalid Expression}$

If either operand has a fractional value (e.g., 5.5 or $10/3$), the expression must be treated as invalid and the program must output **Invalid Expression**.

- **Error Handling:**
 - Syntactic errors (e.g., $10 + * 5$)
 - Lexical errors (invalid tokens)
 - Runtime errors (e.g., division or modulo by zero)

All such cases must output: **Invalid Expression**.

- **Result:** Output the final numerical result as a floating-point value with exactly two decimal places.
- **Precision and Rounding Rule** All intermediate calculations must be performed using full precision (no rounding during evaluation in intermediate steps). Rounding must be applied **only once** to the final result, which must be displayed as a floating-point number with exactly two decimal places.

3 Sample Input/Output Cases

Example 1: Standard Expression

Input: 10 + 5 * 2

Output:

Token Table:

Lexeme: 10	Token: NUM
Lexeme: +	Token: PLUS
Lexeme: 5	Token: NUM
Lexeme: *	Token: MULT
Lexeme: 2	Token: NUM

Result: 20.00

Example 2: Use of Parentheses

Input: (10 + 5) * 2

Output:

Token Table:

Lexeme: (	Token: LPAREN
Lexeme: 10	Token: NUM
Lexeme: +	Token: PLUS
Lexeme: 5	Token: NUM
Lexeme:)	Token: RPAREN
Lexeme: *	Token: MULT
Lexeme: 2	Token: NUM

Result: 30.00

Example 3: Invalid Expression

Input: 10 + * 5

Output:

Token Table:

Lexeme: 10	Token: NUM
Lexeme: +	Token: PLUS
Lexeme: *	Token: MULT
Lexeme: 5	Token: NUM

Invalid Expression

4 Additional Test Cases for Precision

Test Case 1

Input: $(10/3) + 3$

Explanation:

- $10/3 = 3.333333\dots$
- $3.333333\dots + 3 = 6.333333\dots$

Output:

6.33

Test Case 2

Input: $(10/3) * 3$

Explanation:

- $10/3 = 3.333333\dots$
- $3.333333\dots * 3 = 10.0$

Output:

10.00

Test Case 3

Input: $3.33 * 3$

Explanation:

- The value 3.33 is already a rounded input (not computed during evaluation)
- $3.33 * 3 = 9.99$

Output:

9.99

Test Case 4

Input: $(5/2) + (7/3)$

Explanation:

- $5/2 = 2.5$

- $7/3 = 2.333333\dots$

- Sum = 4.833333...

Output:

4.83

Test Case 5

Input: $10 + 5/2 * 3$

Explanation:

- $5/2 = 2.5$
- $2.5 * 3 = 7.5$
- $10 + 7.5 = 17.5$

Output:

17.50

Test Case 6

Input: $1/3 + 1/3 + 1/3$

Explanation:

- Each $1/3 = 0.333333\dots$
- Sum = 0.999999...

Output:

1.00

Test Case 7

Input: $0.33 + 0.33 + 0.33$

Explanation:

- $0.33 + 0.33 = 0.66$
- $0.66 + 0.33 = 0.99$

Output:

0.99

5 Submission Guidelines

5.1 File Naming Convention

- **Lexer File:** Name your LEX source file exactly as `<rollnumber>_lex.l` (e.g., `2411AI09_lex.l`).
- **Parser File:** Name your YACC source file exactly as `<rollnumber>_yacc.y` (e.g., `2411AI09_yacc.y`).
- **ZIP File:** Submit both files in a compressed ZIP named `<rollnumber>_assignment_4.zip`.
- **Numeric Formatting:** Results must support implicit multiplication (like $2 * (3 + 4)$) and output every arithmetic expression as a float with 2 decimal places.
- **Input Support:** Accept both integer and float numbers in input.

5.2 Code Header Requirements

Every file must include a comment at the very top containing your **Full Name** and **Roll Number**.

5.3 Submission Format

- **File Type:** Only `.l` and `.y` files are accepted.
- **Dependencies:** Your code must run using only the standard Flex, Bison, and GCC libraries on the systems.